What DNS is

DNS stands for Domain Name System. It's the internet's address book.

Computers find each other by number. People remember names. DNS is the system that turns one into the other, and it's spread across millions of machines rather than kept in one place.

DNS isn't one thing. It's a set of pieces working together: the names themselves, the companies that keep the lists, the servers that answer questions, and the copies that get cached along the way. Each has its own section on this sheet.

```
google.com → 142.250.72.14  — that swap, done a few trillion times a day
```

Part 1 · Buying a domain (example.com)

Names are read backwards

Computers find each other by number — an IP address like `142.250.72.14`. A domain name is a label pointing at one.

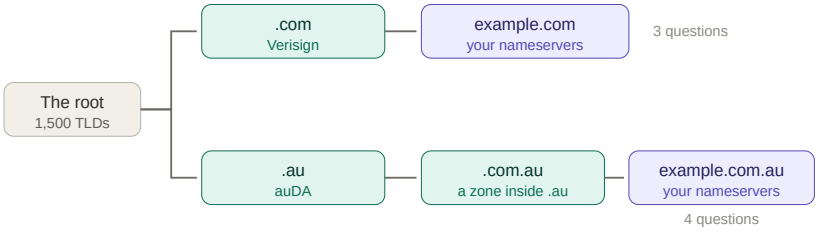
Names are read **right to left**, widest part first. Every dot is one step down a tree.

```
www  .  google  .  com  .
 │        │         │   └─ the root
 │        │         └─ top level
 │        └─ the bit you pay for
 └─ yours to invent, as many as you like
```

There are a few hundred top-level domains (`.com`, `.org`, `.au`). You rent one name inside one of them. Everything to the left of it is yours to create at no cost: `blog.example.com`, `api.example.com`.

What about example.com.au?

It's an extra layer, and a completely separate branch. `.com.au` has nothing to do with `.com` — they just reuse the same word, like two different folders each containing a subfolder called *docs*.



Only two things change: your signpost sits in `.com.au` rather than `.com`, and the registry is auDA rather than Verisign. Zone files, records, nameservers and caching all behave exactly as described elsewhere.

It can go deeper still. In `jobs.careers.vic.gov.au`, `.au` belongs to auDA and `.gov` is handed to the Digital Transformation Agency. Same idea, repeated.

> **Two practical differences.** auDA has eligibility rules that `.com` doesn't — you generally need an Australian presence, usually an ABN. And your registrar has to be accredited for `.au` specifically; not all of them are.

What actually happens when you pay

You
the registrant
Pick a name nobody has taken, and say which nameservers should be in charge of it.

→

Registrar
Namecheap, GoDaddy
A shop licensed to write to the registry for you. This is who you pay, and who you tell when you want to change nameservers.

→

.com registry
Verisign
One company runs the master list for each top-level domain, and adds your signpost to it. Never sells to the public.

→

Your nameservers
Cloudflare, Route 53
The machines your signpost points at. They hold your records and answer questions about your name.

What you pay for is one **signpost** in the `.com` list, reading *anything ending in example.com — go ask these nameservers.* Stop paying and it's taken down: the name still exists, it just leads nowhere.

What your nameservers hold

A list of records, called the **zone**. Each record answers one kind of question about your name.

Record	Answers the question
A	What address is this name at? `example.com → 93.184.216.34`
AAAA	Same question, newer style. Most names carry both. `example.com → 2606:2800:220::1`
CNAME	Is this just another name for something else? `www.example.com → example.com`

Record	Answers the question
NS	Which nameservers are in charge here? `example.com → ns1.cloudflare.com`
MX	Where should email to this domain be delivered? `example.com → mail.google.com`
TXT	Free text. Mostly used to prove you own the name. `"google-site-verification=x7Kd2..."`

Two zone files can exist. Only one is read.

Say you buy `example.com` at Namecheap, then build your site and host it on Cloudflare. During setup Cloudflare tells you to change your nameservers to `ada.ns.cloudflare.com`. Hosts ask for this so they can manage your DNS for you, switch on HTTPS automatically, and route visitors through their own network.

So you paste those names into Namecheap's nameserver box and save. You now have **two** zone files, both real, both editable: the one Namecheap made when you bought the name, and the one Cloudflare made when you signed up.

The `.com` registry's list

Verisign · the signpost

example.com	NS	ada.ns.cloudflare.com
	glue	203.0.113.10

Set in the **nameserver box at your registrar**. This one line decides everything to the right.

The `glue` line is a detail you can ignore. It's only there when a domain's nameservers are named inside itself, the way `google.com` uses `ns1.google.com`. Point at Cloudflare and you'll never need one.

→ names one of these

read by everyone

Zone file at Cloudflare

your web host

example.com	A	93.184.216.34
www	CNAME	example.com
example.com	MX	mail.google.com

ignored

Zone file at Namecheap

where you bought the name

example.com	A	198.51.100.7
example.com	MX	mail.namecheap.com

Still exists. Still lets you edit and save. Nobody asks it anything, so nothing you type here has any effect.

The NS line at the registry decides which one the world reads. It names Cloudflare, so every question about your domain goes to Cloudflare, and Cloudflare's zone file answers. Namecheap's copy is still sitting there, still shows your records, still saves your edits. Nobody ever asks it anything.

> **"I changed the record and nothing happened"** — check which nameservers the registry actually names.

Who holds what

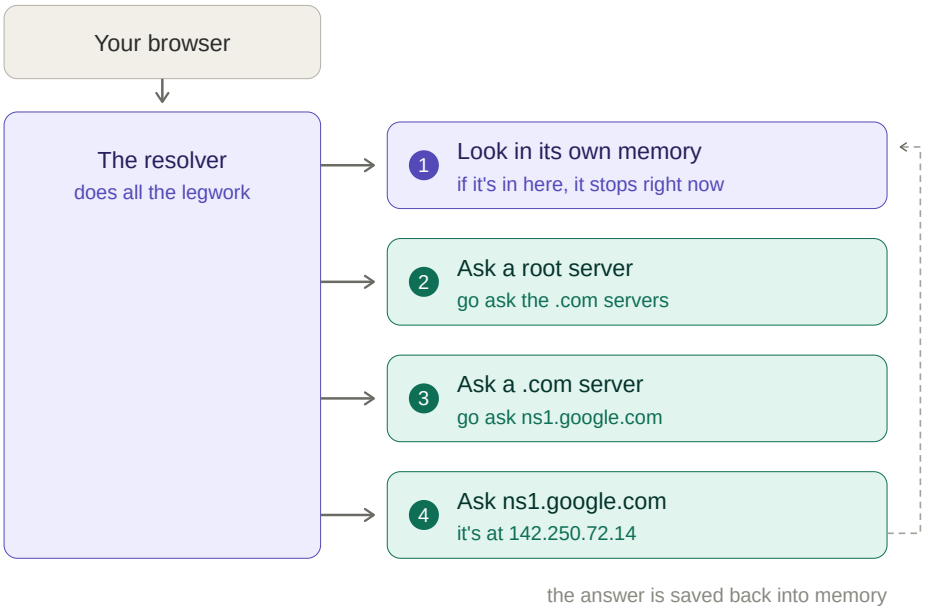
	Registrar	DNS host	App host	What you actually did
1	Namecheap	Namecheap	DigitalOcean	Rented a server, pasted its IP into an A record at Namecheap
2	Namecheap	Cloudflare	Cloudflare	Cloudflare gave you two nameserver names; you pasted them into Namecheap's nameserver field
3	Namecheap	DNSimple	DigitalOcean	DNSimple gave you four nameserver names; you pasted those into Namecheap's field instead

The registrar column never changes — that's just where you bought the name. Only the middle one ever moves.

> Row 1 pastes into a **record**, inside Namecheap's zone file. Rows 2 and 3 paste into the **nameserver field**, which isn't a record at all — it goes to the registry and decides whose zone file gets read. Compare rows 1 and 3: same server, same site, different DNS host.

Part 2 · From google.com to an address

Your computer asks one server, not many



Your browser sends one question to one machine — a **resolver**, run by your internet provider, or a public one like `1.1.1.1`. It does the whole hunt and comes back with a single answer.

Your computer never talks to the servers on the left.

It happens in that order, not all at once. The resolver can't ask the `.com` servers until the root tells it who they are, and can't ask `ns1.google.com` until `.com` names it. Each reply is what makes the next question askable.

What each level actually stores

The root reads the name right to left and only looks at the last part. Its whole list is the top-level domains — who runs `.com`, who runs `.org`. It has never heard of "google". The `.com` servers hold one line per `.com` domain, naming its nameservers.

The root's list
one line per top-level domain · about 1,500 lines

```
com
a.gtld-servers.net

org
a0.org.afilias-nst.info

au
q.au
```

Only reads the last part of the name.

→

The `.com` registry's list
one line per .com domain · about 160 million lines

```
google.com
ns1.google.com

example.com
ada.ns.cloudflare.com

netflix.com
ns-81.awsdns-10.com
```

The list from Part 1. Every line is a signpost someone pays for.

→

`google.com`'s zone
everything about this one name

```
google.com  A
142.250.72.14

google.com  MX
smtp.google.com

mail  CNAME
googlemail.l.google.com
```

The only list with a real address in it.

Reception: "Don't know Sarah, but she's on floor 4."

Floor 4: "Don't know her either. Try the design team."

Design team: "Desk 12, by the window."

One name, different answers. Big sites reply with a different address depending on where you're asking from, so you get sent to a machine near you. Two people, two answers, both correct.

```
google.com from Sydney → 142.250.66.238
google.com from London → 142.250.200.14
```

Each level knows only who to ask next. That middle list is the registry from Part 1: the signpost you rent is one line in it, and this is where it gets read.

Part 3 · Why it's fast, and why changes take time

What happens to a saved answer

`10:00`

First person asks

Memory is empty, so the resolver does the full walk: root, then `.com`, then your nameservers.

slow · ~200ms

→

`10:00 – 11:00`

Everyone else asks

Handed the saved copy straight from memory. No walk at all, however many people ask.

instant

→

`11:00`

The hour is up

The copy is thrown away, and that is **all** that happens. Nothing is fetched — the resolver just has nothing to hand out.

→

`11:07`

Next person asks

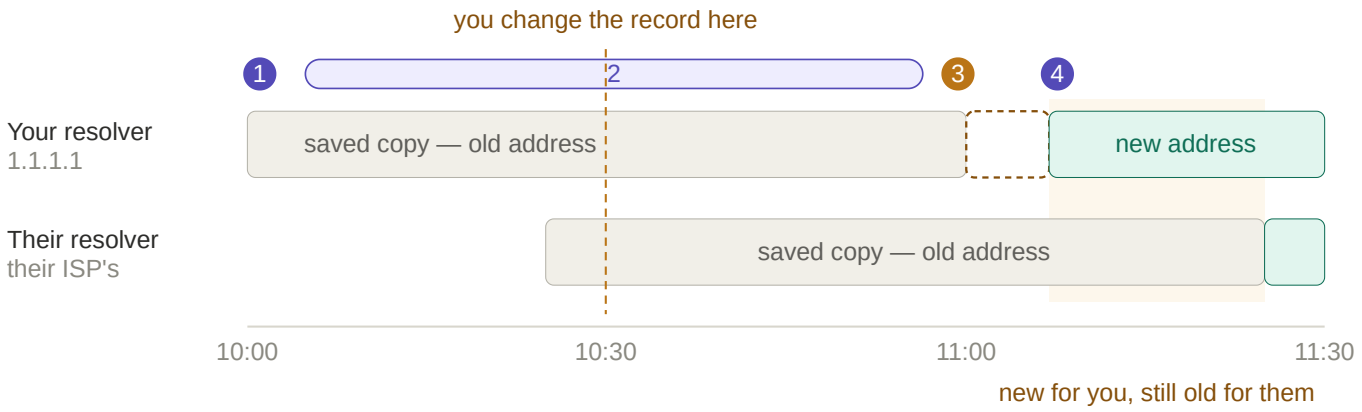
Full walk again, and this time it reads whatever your zone says *now*. Your new address. Saved for another hour.

slow · ~200ms

That hour is the record's **TTL** — a number you set yourself, on each record. One hour is just this example; Google's A record is 300 seconds.

Every resolver has its own clock

The same four steps, plus a second resolver that happened to start its hour 25 minutes later.



"Propagation" isn't really a thing. Nothing gets pushed out across the internet. You're waiting for other people's copies to run out the clock. Drop the TTL the day *before* a change and the wait shrinks from an hour to minutes.

Try it yourself

```
$ dig +short google.com
142.250.72.14

$ dig +short NS google.com
ns1.google.com.
ns2.google.com.
```